

Filter Lock

```
class Filter implements Lock {
    int[] level;
    int[] victim;
    public Filter(int n) {
        level = new int[n];
        victim = new int[n]; // use 1..n-1
        for (int i = 0; i < n; i++)
            level[i] = 0;
    }
    public void lock() {
        int me = ThreadID.get();
        for (int i = 1; i < n; i++) { //attempt level 1
            level[me] = i;
            victim[i] = me;
            // spin while conflicts exist
            while (( $\exists k \neq me$ ) (level[k] >= i && victim[i] == me)) {}
        }
    }
    public void unlock() {
        int me = ThreadID.get();
        level[me] = 0;
    }
}
```

Лемма: Filter-Lock удовлетворяет mutual-exclusion

Доказательство: Пусть изначально lock() вызвали w потоков ($w \leq n$).

Докажем, что после каждой итерации внутри lock(), на следующую итерацию переходит $\max(1, w-L)$ поток.

База ($L = 1$):

- $\exists k \neq i : \text{level}[k] \geq 1$ верно, если хотя бы ещё один поток успел исполнить $\text{level}[i] = 1$
- т.е. при $w = 1$ перейдёт один поток
- $\text{victim}[1] == i$ ровно для одного потока, который последним присвоил свой id,
- значит на следующую итерацию перейдёт $w-1$ поток.

Индукция ($(L = j) \Rightarrow (L = j+1)$)

- пусть верно, что в итерацию ($L = j$) пришёл $w-j$ поток,
- $\exists k \neq i : \text{level}[k] \geq 1$ верно, если хотя бы ещё один поток успел исполнить $\text{level}[i] = 1$
- т.е. при $w = 1$ перейдёт один поток
- $\text{victim}[j] == i$ ровно для одного потока, который последним присвоил свой id,
- значит на следующую итерацию перейдёт $w-j-1$ поток.

Значит, начиная с итерации $L = w-1$ и до $L = n-1$ будет переходить ровно один поток. В из lock() выйдет ровно один поток.

Рассмотрим последующий вызов unlock(). Он снимет с ожидания $w-1$ поток, они перейдут каждый на одну итерацию дальше и снова будут ждать. А самый последний тред выйдет из lock() □

Лемма: Filter-Lock обладает starvation-free

Доказательство: Обратная индукция по уровням итерации.

База ($n-1$): тривиально, содержит не более одного потока Индукция ($j+1 \Rightarrow j$):

Пусть каждый поток достигший $j+1$ входит в критическую секцию. Предположим, поток A застрял на уровне j , как только A устанавливает `level[A]` равным j , то любой поток на уровне $j-1$, который впоследствии считывает `level[A]`, не может войти на уровень j . В итоге, больше потоки не будут входить на уровень j с более низких уровней.

Теперь мы рассуждаем методом индукции о количестве потоков, застрявших на уровне j .

База: A - единственный поток на уровне j или выше, то очевидно, что он перейдет на уровень $j + 1$. Пусть на уровне j не может застрять менее k потоков. Предположим, что потоки A и B застряли на уровне j :

- A застрял до тех пор, пока не будет прочитано значение `victim[j] = A`,
- B застрял до тех пор, пока не будет прочитано значение `victim[j] = B`

Поле `victim` не изменяется и не может быть равно одновременно A и B , поэтому один из двух потоков перейдет на уровень $j+1$, уменьшив количество застрявших потоков до $k-1$, что противоречит индукционной гипотезе. □